

PART IV — FROM RAG TO AGENTIC RAG

Chapter 22

Observability: Building a Full Dry-Run Trace

“An agent you cannot watch think is an agent you can only guess about.”

Chapter 22 — Observability: Building a Full Dry-Run Trace

Every mechanism in Chapters 18 through 21 — the phase state machine, the retry budgets, the multi-verdict judge, the relevance gate — was diagnosed, built, and fixed by someone reading a log. Not a metrics dashboard, not a sampled trace in a hosted observability product: a plain-text `.debug.log` file, dense enough that a specific wrong chunk, a specific malformed verdict, or a specific runaway retry is visible in it directly, without inference.

This chapter builds that log. Not as an afterthought bolted onto a finished pipeline, but as the same kind of first-class artifact the answer itself is — because in this project, it was: bugs from Chapter 18's premature-compression guard through Chapter 20's missing-gate finding were all found by reading trace output first and working backward to the code, never the other way around.

22.1 Why observable agents beat "magic" agents

Definition — Dry-run trace

A complete, human-readable log of everything an agent run did — every prompt sent, every tool called, every score computed, every verdict reached — captured at the moment it happened, so a failure can be diagnosed by reading rather than by re-running with guesses about what to instrument next.

A RAG pipeline that only exposes its final answer is a black box with a good user interface. When that answer is wrong, the only available question is "why," and the only available method is to re-run the query while guessing what to add a ``print`` statement to — which is itself a bet that the failure will reproduce identically on the second try, a bet that is often wrong for anything involving a nondeterministic model call.

An observable agent makes the opposite trade: pay the (cheap, DEBUG-level, disk-only) cost of logging everything on every run, so that when a run does go wrong, the answer to "why" is already sitting in a file. Every named bug across Chapters 18 through 21 that had a "Found Date" and a debug-log filename attached to it in this project's own records was found this way — by reading, not by re-instrumenting.

"Magic" is not a compliment applied to an agent's behavior; it is an admission that nobody watching it can currently explain what it just did. A demo that works is not evidence a pipeline is reliable — it is one sample, and a nondeterministic model call means the next run is a different sample from the same distribution, possibly a much worse one. Observability does not make an agent more reliable by itself. It makes every failure that does occur legible enough to actually fix, which over enough runs is the difference between a project whose defect count trends downward and one that re-discovers the same class of bug indefinitely because nobody could ever quite reproduce it.

22.2 Step-numbered console output

The trace is organized as a sequence of numbered, boxed headers — `STEP 1 — USER QUERY RECEIVED`, and onward through the pipeline — so a reader scanning a long log can locate a specific stage of a specific run without parsing every line between headers.

```
=====
STEP 1 — USER QUERY RECEIVED
=====
Input : "What is an adjustable speed drive and how does it control motor
speed?"
Length: 70 chars
=====
```

A heavy rule (`=`) above and below each header, a thin rule (`-`) around each sub-block within a step — Chapter 14.15's delimiter discipline applied to logs instead of prompts. The visual weight is deliberate: a human eye scanning a terminal or a log file finds a doubled rule far faster than it finds an ordinary line of text, which is the entire value of the convention.

22.3 Logging the embedding step

Chapter 11.1 introduced `\_embed\_and\_log` as the retriever's own record of what a query vector actually looked like. The dry-run trace is where that logging pays for itself directly — a wrong or empty embedding is visible in the trace at the exact moment it happens, not inferred later from a retrieval that returned nothing.

```
=====
STEP: QUERY → EMBEDDING
=====
Query       : "adjustable speed drive"
Model       : all-MiniLM-L6-v2
Shape       : (384,)
First 8 vals : [0.011542, 0.063031, 0.005661, ...]
L2 norm     : 1.000000 (1.0 = fully normalised)
=====
```

An L2 norm that isn't 1.0 is the single fastest thing to check when retrieval quality looks wrong for no visible reason — it means the embedding model changed, or normalization broke, before a single similarity score was ever computed.

22.4 Per-retrieval logging

Every retrieval call logs its ranked results in full — score, source file, and a content preview per chunk — so a reader can judge retrieval quality against the actual question without re-running the query themselves.

```
Merged & ranked 5 doc(s) after dedup/threshold filter:
  Doc 0 — score: 0.8260  [..\data\Adjustable Speed Drive.txt]
    preview: Adjustable Speed Drives provide precise control over
motor speed...
  Doc 1 — score: 0.8153  [..\data\Adjustable Speed Drive.txt]
    preview: Ultimately, Adjustable Speed Drives are not merely
devices for...
```

This is the same score-band vocabulary Chapter 11.4 established — 0.7+ strong, 0.4–0.7 related — made legible at a glance across every chunk a run actually touched, which is what turns "the answer seems off" into "chunk 1 scored 0.815 but is barely on-topic, and here is exactly why."

22.5 Serializing the messages list

Before every model call, the trace prints the entire message list the model is about to receive — not a summary, the actual payload, role by role.

```
=====
FULL MESSAGE SENT TO LLM  [Iteration 1 / max 6]
=====
[0] role=system
    content: You are a research assistant. Answer ONLY from retrieved
chunks...
[1] role=user
    content: What is an adjustable speed drive and how does it control
motor speed?

Tool schemas available: ['retrieve_documents', 'check_answer_quality']
=====
```

This is the ground truth for every question Chapter 18.9's message-scrubbing mechanism raises — did the placeholder actually replace the raw chunk text, does the `tool\_call\_id` pairing actually hold, is the system prompt actually what `\_build\_system\_prompt` was supposed to produce this run. A serialized message list answers all three by inspection, with no assumption that the code did what it was supposed to.

22.6 Per-iteration tool-call logs

Every tool call the model requests is logged twice — once as the model's raw request, once as the orchestrator's dispatch of it — so a reader can see both what the model asked for and what actually ran.

```
tool_calls requested (3):  
  → name : retrieve_documents  
    id   : h8qzjxhdv  
    args : {"query": "adjustable speed drive", "top_k": 5}
```

```
AGENT ACTION — calling tool: retrieve_documents
```

```
args: {"query": "adjustable speed drive", "top_k": 5}
```

Logging the request and the dispatch as two separate blocks, rather than one, is deliberate — Chapter 18.4's args-mutation pitfall is exactly the gap between what the model asked for and what actually executed, and a trace that only logs one side of that gap cannot catch a mutation happening between them.

22.7 Chunks versus retrieval calls — a distinction worth keeping straight

``total_retrievals`` and the length of ``accumulated_document_chunks`` are two different counters tracking two different things, and a trace — or a stats line built carelessly from one instead of the other — can silently conflate them. A single ``retrieve_documents`` call increments the retrieval counter by exactly one, regardless of whether it returns five chunks, one chunk, or zero.

Common pitfall — Reporting one counter as if it were the other

"3 retrievals" and "3 chunks" look interchangeable in a casual log line and are not: three calls that each returned one surviving chunk after threshold filtering leave the agent with three chunks total and three calls spent, while one call that returned five chunks leaves it with five chunks and one call spent — the same visible number, two entirely different retrieval budgets consumed. Label every count in a trace with what it is actually counting — ``total_retrievals=`` and ``accumulated_documents=`` as separate, explicitly named fields — rather than a bare number a reader has to guess the meaning of.

The distinction matters most exactly where Chapter 18.5's budget arithmetic lives: ``MAX_TOTAL_RETRIEVALS`` caps calls, not chunks, so a trace that only reports chunk counts cannot explain why the loop stopped retrieving with plenty of room left in the answer's evidence.

The same ambiguity can hide inside a single word in a status message. "Retrieved 5 results" is genuinely unclear without context — five chunks from one call, or one chunk apiece from five calls, read identically in casual phrasing and mean very different things for how close the run is to its retrieval ceiling. A trace built for debugging should never make a reader do that translation from memory; every count it prints should name, in the same line, exactly what it counted.

22.8 The [CONTEXT SIZE @ iter N] telemetry block

After every model call, the trace reports exactly how large the payload that was just sent actually was — message count, character count, an estimated token count, and the real prompt-token count the provider's own response reported.

```
[CONTEXT SIZE @ iter 2]
  messages in payload      : 6
  total content chars      : 6,389
  ~tokens (chars/4 est.)  : 1,597
  actual prompt tokens     : 1,724 (from API)
  cumulative prompt tok   : 2,544
```

Two numbers are worth comparing every time: the cheap chars/4 estimate against the real, API-reported token count. When they diverge substantially — code-heavy or non-English content typically pushes real tokens well above the character-based estimate — that divergence itself is a signal Chapter 23's token-budget planning needs to account for, not an error to explain away.

22.9 The [CTXSIZE] greppable log line

Every '[CONTEXT SIZE @ iter N]' block is immediately followed by a second, single-line, machine-parseable restatement of the same numbers.

```
[CTXSIZE]          iter=2          msgs=6          chars=6389          prompt_tokens=1724
cum_prompt_tokens=2544
```

The human-readable block above it exists for a person reading the log top to bottom during a single debugging session. This line exists for 'grep', 'awk', or a small offline script pulling every '[CTXSIZE]' line out of a hundred stored logs to plot prompt-token growth across a whole benchmark run — the same telemetry, in a second format chosen for a second, entirely different consumer. Logging one fact once, in whichever format is easiest to write, optimizes for the person writing the log line and against everyone who has to read it later at scale.

22.10 Reading a real dry-run trace from start to finish

Reading a trace in order tells a coherent story, not a pile of disconnected log lines. A real run: 'STEP 1' receives the question. The full message list is serialized and its '[CONTEXT SIZE]' reported. The model responds with three parallel 'retrieve_documents' calls. Each dispatches through 'AGENT ACTION', embeds via 'STEP: QUERY → EMBEDDING', and returns five scored, sourced, previewed chunks. 'VALIDATE-RETRIEVAL' judges them PASS at 3-of-5 relevant and names which two were dropped and why. Two surviving chunks are flagged as near-duplicates by cosine similarity, 'CHUNK MERGE' consolidates them, and 'VALIDATE-MERGE' confirms the merge FAITHFUL with zero fabricated or dropped claims.

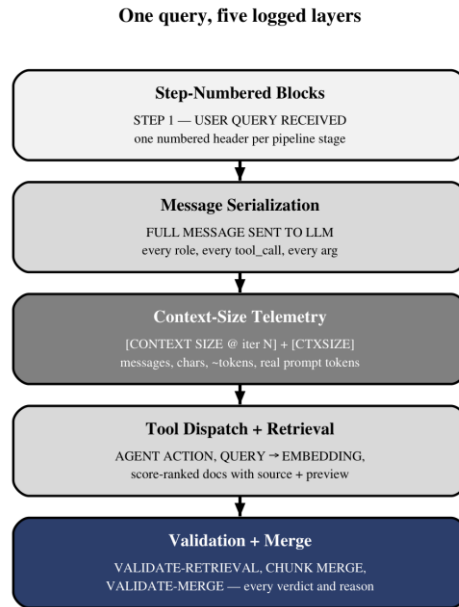


Figure 22.1 — Five layers, read top to bottom, reconstruct exactly what one query actually did.

Figure 22.1 is that same story compressed into five bands. Reading a trace well is reading it in this order every time — structure first (which step, which iteration), then payload (what was actually sent), then cost (how big was it), then action (what did the model ask for and what ran), then judgment (what did each validator decide, and why). A reader who jumps straight to the final answer and works backward only when it looks wrong is reading the trace as a last resort; a reader who reads it this way by habit usually catches the wrong chunk before it ever reaches the answer at all.

22.11 Turning trace output on and off cleanly

None of this detail costs anything in production by default, because it never reaches the console. `setup\_logging` wires two handlers to the same logger hierarchy at two different thresholds: a console handler defaulting to `logging.INFO`, and a file handler fixed to `logging.DEBUG`, unconditionally, regardless of what the console is showing — Figure 22.2 traces both paths from the same call site.

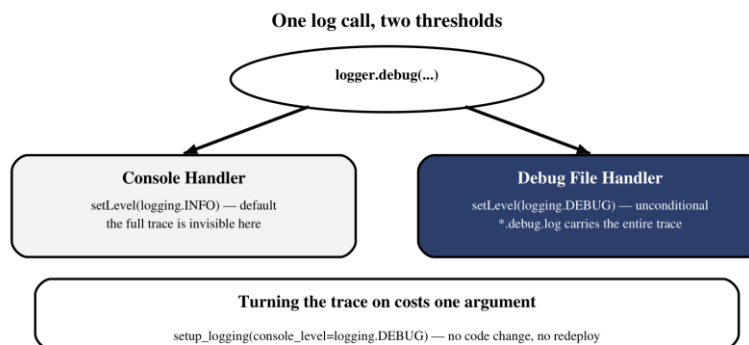


Figure 22.2 — The same log call reaches both handlers; only the threshold decides which one keeps it.

```

def setup_logging(log_dir=DEFAULT_LOG_DIR, app_name=None,
                  console_level=logging.INFO):
    console_handler.setLevel(console_level) # quiet by default
    debug_handler.setLevel(logging.DEBUG)   # the full trace, always,
to disk
  
```

Every `logger.debug(...)` call in this chapter's examples is filtered out of the console by default and written to `{app_name}_{timestamp}.debug.log` regardless — the trace is always being captured, whether or not anyone is watching it live. Turning it on for a live session costs exactly one keyword argument, `setup_logging(console_level=logging.DEBUG)`, no code change and no redeploy — the same discipline Chapter 20's severity-level audit enforced in the other direction, making sure a message's assigned level actually matches how urgently a human needs to see it.

A trace this detailed is not free — it is disk space, and it is a discipline every new log line has to earn by being written at the right level with the right label. What it buys in return is the thing every chapter since Chapter 18 has depended on without saying so directly: every bug in this book that has a root cause instead of a guess was found by someone reading exactly this kind of log. Chapter 22B goes one level deeper into the busiest part of that trace — the compression pipeline's own NAC, DC, and LBC stages, and the semantic reasoning behind why retrieved context needs compressing at all.